

TD&TP 8 : LES FONCTIONS EN PARAMÈTRES

Pourquoi des fonctions en paramètre

Lorsque l'on crée certaines structures, en C, ainsi que les fonctions qui s'y rattachent, on aimerait pouvoir réutiliser ses fonctions pour des données de n'importe quel type.

Par exemples, un tasMin diffère d'un tasMax par la fonction de comparaison. Un tri en ordre croissant ou décroissant d'une liste de données ne diffère que par la fonction de comparaison.

Le problème est que certaines fonctions (telles celles de comparaison) sont propres au type de données.

Ces fonctions doivent donc être des paramètres.

Le passage d'une fonction en paramètre se fait par un passage de pointeur sur fonction

Exemple :

Fonction `OperatBinaire(opd1, opd2, operation)` qui applique un calcul (ex somme, produit, pgcd) aux arguments `opd1` et `opd2`.

/ f est un pointeur vers fonction */*

```
int (*f)(int,int);
```

Pour indiquer que le symbole `f` est un pointeur (`*f`) vers une fonction à deux paramètres (`int, int`) rendant un entier `int`.

Le signe `*` est nécessaire pour indiquer que `f` est pointeur.

```
int OperatBinaire(int x, int y, int (*f)(int, int) ){  
    /*appel de f sur les arguments x et y*/  
    return ((*f)(x,y) );  
}
```

Pour l'appel ou la référence à ce paramètre formel `f`, on écrit `(*f)(x, y);`

i.e. appel de la fonction pointée par `f` sur les paramètres `x` et `y`. Là aussi on utilise le signe `*` et les parenthèses.

```
int somme(int a, int b){  
    return(a+b);  
}  
  
int diff(int a, int b){  
    return(a-b);  
}  
  
int produit(int a, int b){  
    return (a*b);  
}  
  
// b !=0  
int division(int a, int b){  
    return (a/b);  
}
```

En d'autres termes, `OperatBinaire(x, y, f)` est une fonction qui a pour résultat `f(x, y)` où `f` est variable.

Appel

```
void main(void){  
    printf(« %d »,OperatBinaire(3, 4, somme) );  
    printf(« %d »,OperatBinaire(3, 4, diff) );  
    printf(« %d »,OperatBinaire(3, 4, &produit) );  
    printf(« %d »,OperatBinaire(3, 4, &division) );  
}
```

Remarques

- Noter qu'il n'est pas fait usage de l'écriture `&somme` ou `&produit` dans l'appel. Le compilateur comprendra, vu que ce sont des fonctions, qu'il doit passer leur adresse en paramètre. Voilà pour ce qui est appel et transmission des paramètres effectifs.
- La présence des parenthèses (`*f`) qui sont nécessaires, car l'écriture `int *f()` est la déclaration d'une fonction rendant un pointeur sur un entier.

Tableau de fonctions

Exemple : soient deux nombres entiers et un caractère représentant un opérateur binaire dont on admet qu'il a une valeur parmi '+', '-', '*', '/', '%'. Effectuer l'opération correspondant au signe choisi et afficher le résultat.

```
#define DIM 5
```

On utilisera un tableau d'opérateurs c-à-d un tableau de (pointeurs vers) fonctions rendant un entier :

```
int (*tabOperat[DIM])();
```

(*tabOperat[i]*)(*x,y*); est alors l'appel de la ième fonction du tableau sur les paramètres *x* et *y*. On initialisera le tableau par :

```
int (*tabOperat[DIM])() = { somme, diff, prod, division, modulo};
```

```
Void main(void){ //Version 1
int (*tabOperat[DIM])() = { somme, diff, prod, division, modulo};

printf("Entrez le premier entier : ");
scanf(" %d",&i);
printf("\nEntrez le deuxième entier : ");
scanf(" %d",&j);
printf("\nEntrez la fonction 1 somme 2 diff 3 prod 4 quotient
5 modulo : ");
scanf(" %d",&choix);
if ((choix > 0) && (choix <= 5))
    printf("\n Résultat : %d.\n", (*(tabOperat[choix-1])) (i,j));
else
    printf("\n Mauvais numéro de fonction.\n");
}
```

Renommage

// le type *fptr* est celui d'un pointeur de fonction à 2 paramètres de type *int* et de valeur de retour *int*

```
typedef int (*fptr)(int, int);
```

// *fptr tabOperat[DIM]* à la place de *int (*tabOperat[DIM])()*(*int*, *int*)

```
fptr ftab[DIM] = {somme, diff, prod, division, modulo}; //tableau statique
```

```
fptr *ftab; //tableau dynamique
```

```
int (*tabOperat[DIM])() = { somme, diff, prod, division, modulo};
```



```
fptr ftab[DIM] = { somme, diff, prod, division, modulo };
```

```
// fptr ftab[DIM] = { &somme, &diff, &prod, &division, &modulo };
```

Version 2

/* définition d'une fonction de calcul selon un pointeur de fonction passe en paramètre avec les paramètres correspondants*/

```
int calcul (fptr f, int a, int b){
    return (*f)(a, b);
}
```

```
void main(void){
int (*tabOperat[DIM])() = { somme, diff, prod, division, modulo};

printf("Entrez le premier entier : ");
scanf(" %d",&i);
printf("\nEntrez le deuxième entier : ");
scanf(" %d",&j);
printf("\nEntrez la fonction 1 somme 2 diff 3 prod 4 quotient
5 modulo : ");
scanf(" %d",&choix);
if ((choix > 0) && (choix <= 5))
    printf("\n Résultat : %d.\n", calcul (tabOperat[choix-1], i,
j));
else
    printf("\n Mauvais numéro de fonction.\n");
}
```

Recherche d'un Élément « elt » dans un Tableau « tab » compose de *n* éléments (TElement est le type des éléments du tableau)

Cet algorithme est général et marche pour n'importe quel tableau pourvu qu'on lui fournisse une fonction, comp ici, qui compare deux éléments entre eux et qui doit donc dépendre du type des éléments du tableau. Voici son implantation en C:

```
int recherche(int n,TElement* tab,TElement elt,
int (*comp)(TElement,TElement)) {
int i;

//Ajout d'une sentinelle
tab[n]=elt;
//Parcours séquentiel avec arrêt
i=0;
while(! ((*comp)(elt,tab[i]) )
    i++;
return i<n;
}
```

Intérêt des fonctions en paramètres

- Les fonctions en paramètres, comme signalé auparavant, permettent d'écrire des procédures (fonctions C) dites génériques, dans le sens où on écrit un code qui peut être utilisable plusieurs fois, non seulement en ce qui concerne l'exécution d'un même algorithme (abstraction procédurale classique), mais ce code peut donner lieu à des calculs différents.
- En effet on peut définir une fois pour toute des unités de programmes de fonctionnalités génériques telles qu'on puisse les utiliser dans des applications spécifiques.

Considérons l'algorithme de recherche d'un élément e dans un tableau T de taille n et qui retourne un entier égal à l'indice d'apparition de l'élément e dans le tableau, et -1 sinon.

Pour cela on doit comparer (fonction *comp* en paramètre) e avec les éléments de T pris un par un.

Exercice1

Tester la fonction ci-dessus avec des tableaux d'entiers, de complexes, de vaches, etc.

Exercice2

En utilisant la fonction de comparaison, réécrire les implémentations des algorithmes suivants :

- 1) Recherches séquentielles et dichotomiques.
- 2) Tris par insertion, fusion, rapide, etc.